

PROJECTMARKET · FOUNDATION

Tech Stack

Decisions and their rationale

INTERNAL DOCUMENT

2026-05-15

Stack decisions and the reasoning behind them. Each decision lists alternatives considered so the rationale is auditable and revisitable.

Web client

Decision: Next.js (App Router, TypeScript).

Server-rendered. The product surface (listings, profiles, search) is both the indexable surface and the interactive surface; the framework has to handle both natively.

- Why not **Flutter web**: renders to canvas (or to semantic-free HTML), neither of which is indexable. Marketplace discovery via search engines is critical and cannot be retrofitted onto a canvas-rendered app. First-paint times are also poor.
- Why not a **SPA without SSR** (Vite + React, Vue without Nuxt): same SEO problem; large initial bundles hurt first paint for users landing from search.
- Why App Router specifically: server components reduce JS shipped to the client; streaming and partial pre-rendering simplify the SEO + interactive mix.

Styling and components

Decision: Tailwind CSS + shadcn/ui.

Tailwind for utility styling. shadcn/ui for accessible component primitives — its source lives in this repository rather than as a dependency, so it is fully customisable without lock-in.

Alternatives considered: Mantine, Chakra, MUI (heavier runtime, harder to deeply customise); CSS Modules or vanilla CSS (reinvents the design system); Radix UI without shadcn (weaker out-of-the-box coverage).

Database

Decision (preferred, awaiting formal commit): Supabase (Postgres with PostGIS).

Reasons:

- **Geo**: PostGIS handles "Sellers near me" natively; no GeoHash hacks.
- **Relational**: marketplaces are heavily relational (Users ↔ Listings ↔ Transactions ↔ Reviews); SQL joins beat client-side reconstruction.
- **Full-text search**: native in Postgres; no Algolia / Elasticsearch needed at launch scale.
- **JSONB**: flexible attribute storage for concept-specific Listing fields.

- **Bundled services:** Supabase Auth, Storage, Realtime, Edge Functions in one console.
- **EU hosting:** GDPR data-residency requirement satisfiable.
- **Portable:** plain Postgres underneath; nothing locks us in.

Alternatives considered:

- **Firebase / Firestore:** ruled out. No native geo, weak complex queries, cost balloons under read-heavy marketplace traffic, no full-text search, non-portable data model.
- **Self-managed Postgres on Hetzner / Railway:** viable, cheaper at scale, but more ops work — wrong tradeoff for a one-person team in this phase.
- **PlanetScale / MySQL:** no PostGIS equivalent.

Authentication

Decision: deferred to final commit. **Default candidate:** Supabase Auth.

Open questions:

- Email + password, magic link, or password-less only?
- Phone OTP as a first-class signup path? (Common in Romanian marketplaces for trust reasons.)
- OAuth providers: Google, Apple, Facebook — which to enable initially?

Supabase Auth supports all of the above; the choice is which to enable.

File storage

Decision: Supabase Storage initially, with the option to move image hosting to Cloudflare R2 + Cloudflare Images later.

Portfolios and product galleries generate image volume. Supabase Storage covers foundation and early launch. If egress costs rise, Cloudflare R2 has free egress and Cloudflare Images handles transforms cheaply.

Internationalisation

Decision: next-intl.

Externalised strings from day one. RO + EN scaffolding present immediately; RO is the launch locale; EN is in place for international rollout and for the dev team.

Deployment

Decision: Vercel for web hosting.

First-class Next.js integration, automatic edge CDN, EU regions available. Free tier covers foundation phase; paid tier (~\$20/month) covers small-launch traffic.

Alternative: self-host the Next.js app on a VPS or Cloudflare Workers. Cheaper at scale, more setup; revisit when costs justify.

Email (transactional)

Decision: Resend.

Modern API, generous free tier, EU region available. Used for verification emails, transactional notifications, password resets.

Alternatives: Postmark (excellent deliverability, slightly higher cost), AWS SES (cheapest, most setup).

Maps and geocoding

Decision: Mapbox.

Address autocomplete, geocoding, optional map display. More generous free tier than Google Maps Platform for the kind of usage expected; explicit terms allow displaying results on a non-Google map.

Alternatives: Google Maps Platform (more polished, stricter TOS, more expensive), OpenStreetMap-based stacks (cheaper, more setup).

Error monitoring

Decision: Sentry.

Free tier covers low-traffic launches. EU region available.

Analytics

Decision: PostHog (EU-hosted) or Plausible (EU-hosted).

GDPR-friendly out of the box. Avoid Google Analytics for EU compliance reasons.

Mobile (deferred)

Two candidates when mobile becomes the priority:

- **Expo / React Native** — shares TypeScript, Zod schemas, API client, and business logic with the Next.js web app. Lower cognitive load for a JS-fluent team.
- **Flutter** — separate Dart codebase, best-in-class native UX, no UI code shared with web.

Decision deferred until web has traction and mobile is the bottleneck.

Payments (idea catalogue, not foundation scope)

Future direction: **Stripe Connect** for marketplace payments. Carries built-in KYC for Sellers; supports both service and goods flows. Will require careful schema design when added; the foundation reserves space in the data model but does not implement.